The macro expands to a token sequence that can be used to initialize one of the standard atomic types with the specified value in an expression of the following form:

```
std::atomic<type> x = ATOMIC_VAR_INIT(val);
```

The specified value must be compatible with the nonatomic type corresponding to the atomic variable, for example:

```
std::atomic<int> i = ATOMIC_VAR_INIT(42);
std::string s;
std::atomic<std::string*> p = ATOMIC_VAR_INIT(&s);
```

Such initialization is not atomic, and any access by another thread to the variable being initialized where the initialization doesn't happen-before that access is a data race and thus undefined behavior.

D.3.4. std::memory_order enumeration

The `std::memory_order` enumeration is used to specify the ordering constraints of atomic operations.

Declaration

```
typedef enum memory_order
{
    memory_order_relaxed,memory_order_consume,
    memory_order_acquire,memory_order_release,
    memory_order_acq_rel,memory_order_seq_cst
} memory_order;
```

Operations tagged with the various memory order values behave as follows (see chapter 5 for detailed descriptions of the ordering constraints).

Std::Memory_Order_Relaxed

The operation doesn't provide any additional ordering constraints.

Std::Memory_Order_Release